

Manual técnico

Como a aplicação está feita, as decisões que não são óbvias a ler o código, e as armadilhas que já custaram bugs. Para a lista de endpoints, ver [API.md](#). Para instalar e correr, ver o [README](#). Para o manual de utilização, ver [MANUAL.md](#).

1. ARQUITECTURA

web/	controllers + DTOs. Traduz HTTP para domínio e vice-versa.
service/	regras de negócio. É aqui que vive a lógica.
domain/	entidades JPA e os cálculos que lhes pertencem.
repository/	interfaces do domínio + implementações em memória (testes)
repository/jpa	adaptadores Spring Data (produção)
security/	autenticação, autorização, filtro de conta activa
resources/db/migration	Flyway. É o dono do esquema.
resources/static	frontend, sem passo de build

Stack: Java 21, Spring Boot 3.5, PostgreSQL, Flyway, HTML e JavaScript simples.

O padrão dos repositórios

Os serviços dependem de **interfaces no pacote `repository`**, nunca de Spring Data. Há duas implementações de cada uma:

- `*RepositoryImpl` em memória, usada pelos testes, todas assentes em `RepositorioEmMemoria<T>`.
- Um adaptador em `RepositoriosJpa`, que delega num `*JpaRepository`.

Isto é o que permite os testes de serviço correrem sem base de dados nenhuma.

Mantém as duas implementações com o mesmo comportamento, incluindo a ordenação. Já houve fakes a devolver por ordem de inserção onde a consulta JPA real tinha `ORDER BY`, o que deixa um teste passar numa ordem que a produção não dá.

EntidadeBase

Os ids são atribuídos pela aplicação (`UUID.randomUUID()`), não pela base de dados. Sem `EntidadeBase` a implementar `Persistable`, o `save()` do Spring Data via um id preenchido, concluía que a entidade não era nova e fazia `merge` em vez de `persist`. As consequências eram silenciosas: relações em cascata não gravadas e a mesma entidade duas vezes na sessão.

`LigaLogo` é a excepção e não a estende, de propósito: é gravada por um `insert ... on conflict` nativo que nunca passa pelo `save()`.

2. MODELO DE DOMÍNIO

Contas

`Gestor` é a única entidade com credenciais. `Treinador` não tem password.

`Treinador` **não é a pessoa: é o lugar de treinador de uma equipa**. A pessoa é a conta. Uma linha de `Treinador` por equipa — `equipa.treinador_id` é único desde a V9 — e quem treina três equipas tem três linhas, todas a apontar à mesma conta. É por isso que a pergunta "que equipas treino eu" se faz por conta e devolve uma lista de treinadores:

```
treinadorRepository.buscarPorConta(conta).stream()
    .flatMap(treinador -> equipaRepository.buscarPorTreinador(treinador).stream())
```

`Treinador.conta` é um `@ManyToOne` opcional e **não único** para `Gestor`. Daí sai o que interessa:

- um treinador pode não ter conta nenhuma, e está tudo bem;
- a mesma conta pode treinar várias equipas, em ligas diferentes;
- quem gere uma liga e treina uma equipa usa um só login.

`Treinador.nome` é o rótulo que o gestor escreveu, e continua a ser o nome mostrado mesmo depois de haver conta ligada — a lista de equipas é do gestor, e o nome da conta é outra coisa, o da pessoa. Corrige-se com `PATCH .../equipas/{id}/treinador`.

Há dois tipos de convite, com entidades separadas de propósito:

	CONVITE	CONVITETREINADOR
Cria	administrador	gestor dono da liga
Para	conta de gestor	ocupar o lugar de treinador de uma equipa
Aberto?	sim, quem tiver o código escolhe quem é	não, já nasce apontado a uma equipa

Três regras do `ConviteTreinadorService` que valem a pena guardar:

- **Emitir é idempotente.** Se já houver convite por usar para aquele lugar, é esse que volta. Duas credenciais válidas para a mesma equipa é o estado que se quer evitar, não um detalhe de interface — e desde a V11 quem o garante é um índice único parcial em `convite_treinador (treinador_id) where usado_em is null and revogado_em is null`, porque procurar-e-depois-inserir é uma corrida: dois pedidos ao mesmo tempo passavam os dois pela procura. Quem perde a corrida leva um 409 do `GlobalExceptionHandler`, e o botão fica desligado enquanto o pedido corre para não ser preciso chegar lá.
- **Não há convites sem prazo.** `diasValidade` a `null` usa `VALIDADE_OMISSAO_DIAS` (30). Um link destes vive numa conversa de WhatsApp para sempre.
- **Aceitar revoga o que sobrou** (`revogarPendentes`), porque o lugar passou a estar ocupado. A verificação `treinador.temConta()` em `TreinadorContaService` continua lá para o caso de dois pedidos correrem ao mesmo tempo.

Emitir e enviar são duas transações, sempre — no convite de uma equipa e no da liga toda. É por isso que existe o `EmissaoDeConvitesService`, que só emite e devolve registos, e que nenhum dos dois controladores de emissão é `@Transactional`: o convite é gravado primeiro, e só depois é que o email sai. Ao contrário, uma falha a gravar deixava uma mensagem entregue a apontar para um convite que não existe, e um email não se retira depois de sair. É também a razão de o `enviarPorEmail` receber um id: as entidades da transação anterior não sobrevivem ao commit.

No lote, cada envio vai ainda dentro do seu próprio `try`. Os convites já estão emitidos nessa altura, e a resposta é a única cópia dos links que o gestor tem para entregar à mão: uma excepção num deles não pode levar os outros com ela.

A autorização sobre um convite é **pela equipa**, não por quem o emitiu: uma liga pode mudar de gestor (`AlterarGestorRequest`) e o convite continua a ser daquela equipa.

`Treinador.email` é contacto do lugar, dado pelo gestor, e não uma credencial: quem aceita o convite escolhe o email da conta no registo e pode ser outro. Nada liga os dois. O envio é travado por convite — três no máximo, nunca dois a menos de dez minutos —, e conta-se só o que saiu: uma falha do Resend não pode gastar o travão. Revogar e emitir de novo contorna o limite, e é deliberado — dá outra credencial e deixa rasto, não é coisa que se faça sem dar por ela.

`podeCriarLigas`

Só uma conta com esta flag cria ligas próprias. Nasce a `true` no caminho normal de convite de gestor, e a `false` em `TreinadorContaService.registar`. Está imposto no `SecurityConfig`:

```
.requestMatchers(HttpMethod.POST, "/api/ligas").hasAuthority("PODE_CRIAR_LIGAS")
```

`ligar()` (juntar um convite de treinador a uma conta que já existe) **não** mexe na permissão: quem já era gestor continua a ser.

A migração V6 criou a coluna com `default true`, por isso todas as contas anteriores ficaram com a permissão, incluindo contas de treinador. Não há código que corrija isso; é uma limpeza manual na administração.

Jornadas

`EstadoJornada`: `ABERTA` → (`DESEMPATE`) → `FECHADA`. Só pode existir uma jornada não fechada por liga.

As primeiras cinco são `TREINO` e as seguintes `OFICIAL`, e **cada tipo numera a partir de 1**. É daí que vem a armadilha da secção 4.1.

Dívidas

`Divida` tem uma equipa (um para um) e uma lista de `BlocoDivida`. Cada bloco é `INSCRICAO` (uma vez), `PERIODO` (recorrente, pela posição em cada jornada do bloco) ou `CLASSIFICACAO` — as cobranças de época, que levam nome próprio.

`RegraDivida` é opcional, uma por liga. Sem ela não há cobrança automática nenhuma. Com ela, o valor por posição sai de uma de duas escalas (`RegraDivida.valorDaPosicao`):

Fórmula — a de sempre, uma rampa de degraus iguais com tecto:

```
escalao = (posicao - 1) / equipasPorEscalao // divisão inteira
valor   = min(valorInicial + incremento * escalao, valorMaximo)
```

Tabela (`EscalaValor`), uma linha por posição) — para as ligas cuja tabela não é uma rampa regular. A que motivou isto sobe 0,20€ por lugar até ao 7º e 0,10€ daí em diante, e não há fórmula que diga aquilo: obrigá-la a uma era forcer a liga para caber no programa. Abaixo do fim da tabela repete-se a última linha, que é o mesmo que o tecto faz na fórmula — não cobrar nada a quem fica para lá do fim premiava exactamente quem ficou em último.

A tabela chega colada do telemóvel e é lida pelo `EscalaColada`, que aceita as formas que aparecem nessas notas (vírgula ou ponto, com ou sem €, hífen ou travessão) e recusa saltos: não há maneira honesta de adivinhar uma posição que falta. `substituirTabela` reaproveita as linhas em vez de as apagar e recriar — no mesmo commit o Hibernate grava as inserções antes das remoções e as linhas novas esbarram na unicidade (`regra_id, posicao`) das antigas, o que dava 409 ao guardar a mesma regra duas vezes. É a mesma armadilha dos números de bloco, mais abaixo.

O cálculo da fórmula está espelhado em `calcularValorEscalao` no `app.js`, mas só para a pré-visualização. **A cobrança a sério é sempre do servidor.**

Duas coisas que variam de liga para liga e que estavam assumidas no código:

- `RegraDivida.cobraTreino` — se as jornadas de treino entram nos blocos.
- `Liga.pontosTreinoContam` — se os pontos delas contam para a classificação. Vive na liga e não na regra porque é do formato da prova: uma liga sem regra nenhuma continua a ter classificação.

Ambas nascem no comportamento antigo (`true`), para nenhuma liga a decorrer mudar de contas.

`Jornada.versao` faz o mesmo pela jornada, e por uma razão medida: sem ela, dois pedidos simultâneos a fechar a mesma jornada liam-na aberta, passavam os dois pela verificação do estado e cobravam o bloco **duas vezes**. Em cerca de metade das corridas com dois pedidos ao mesmo tempo — e um duplo clique no botão chegava. Agora quem perde a corrida esbarra na versão e leva um 409.

`Divida.proximoNumeroBloco` é persistido e protegido por `@Version`, para duas transacções em simultâneo não gravarem dois blocos com o mesmo número. Na prática quem perde a corrida esbarra primeiro na restrição `bloco_divida_numero_unico`, porque o Hibernate grava as inserções antes das actualizações; ambos os casos dão 409 pelo `GlobalExceptionHandler`.

Cobranças de época

`CobrancaPeriodo` é uma cobrança presa a uma jornada **oficial**, pela posição na **classificação geral** — o "Inverno" e o "Verão" de algumas ligas. Tem tabela própria, acontece uma vez (`cobradaEm`), e o bloco que cria leva o nome, para na dívida se ler "Inverno" e não "Bloco 12".

A jornada é indicada pelo número oficial e não pela contagem corrida desde o início: as de treino podem ser mais ou menos do que se espera, e uma cobrança que escorregue uma jornada é dinheiro cobrado no sítio errado.

Um empate trava a cobrança — mas só quando muda o valor. É a mesma regra que `fecharJornada` já aplica: cobrar com o empate por desfazer era cobrar a mais a umas e a menos a outras, e com o dinheiro lançado não há forma limpa de corrigir. A diferença é que aqui o desempate é da classificação geral, que fora disto não decide dinheiro nenhum e por isso continua ordenada por nome. Um empate entre lugares que pagam o mesmo não pergunta nada a ninguém.

`acertarCobrancas` casa pelo nome em vez de substituir tudo: sem isso, guardar a regra outra vez fazia uma cobrança já feita renascer por cobrar — e ser cobrada duas vezes.

3. OS DOIS FLUXOS QUE MEXEM EM DINHEIRO

3.1 Fecho de bloco

Em `JornadaService.fecharJornada`, depois de a jornada fechar:

1. Se a liga não tiver `RegraDivida`, não acontece nada.
2. Recolhem-se as jornadas **fechadas e ainda não incluídas num bloco** (`Jornada.incluidaEmBloco`).
3. Se forem tantas ou mais do que `jornadasPorBloco`, fecha-se um bloco.
4. Cada equipa activa paga a **soma do seu valor em cada uma dessas jornadas**, pela posição que teve em cada uma.
5. Essas jornadas ficam marcadas, e nunca mais entram noutro bloco.

O passo 5 é o que torna isto seguro. Sem essa marca, mudar `jornadasPorBloco` a meio da época fazia o fecho seguinte olhar outra vez para jornadas já cobradas e cobrá-las a dobrar. E como a selecção é por marca e não por "as últimas N", não depende de ordenação nenhuma.

3.2 Desempate

Se, ao fechar, sobraem equipas com a mesma pontuação, a jornada fica em `DESEMPATE`, **sem cobrar nada**. `resolverDesempate(jornada, ordem)` aplica a ordem dada, atribui posições sequenciais, marca os resultados afectados com `desempateManual` e só então deixa correr o fecho do bloco.

A ordenação primária continua a ser a pontuação; a ordem submetida é só critério secundário, por isso uma lista mal formada nunca troca equipas entre pontuações diferentes. A validação exige que a lista seja exactamente o conjunto empatado.

`inserirResultado` recusa enquanto a jornada está em `DESEMPATE`. Sem isso, mexer nas pontuações invalidava o empate detectado, em silêncio.

4. ARMADILHAS

Todas estas já produziram bugs reais. Valem uma leitura antes de mexer.

4.1 `liga.getJornadas()` não vem por ordem cronológica

A colecção é `@OrderBy("numJornada")`, e `numJornada` reinicia em 1 quando começam as oficiais. Assim que uma liga tem os dois tipos, essa ordem intercala-os.

Usa `Jornada.OrdemCronologica` sempre que a ordem real interessa (tipo primeiro, número depois). Não ordenes por tipo no `@OrderBy`: como o enum é gravado como texto, "OFICIAL" viria antes de "TREINO".

4.2 `listarDividas(liga, estado)` não serve para somar dinheiro

Filtra pelo estado da **dívida da equipa**, que só diz se ainda sobra alguma coisa por pagar. Uma dívida pendente pode ter blocos já pagos lá dentro.

Para dinheiro, usa `listarPorLiga(liga)` e soma **bloco a bloco**, como faz `calcularPoteDaLiga`.

4.3 O estado da dívida é calculado, nunca assumido

`Divida.atualizarEstado()` recalcula a partir dos blocos, e é chamado sempre que algum muda. Não ponhas `setEstado` à mão. Um bloco de valor zero nasce já resolvido; foi por se assumir o estado que uma dívida cujo primeiro bloco era de 0,00 € ficava pendente a dever zero.

4.4 As permissões da sessão são relidas a cada pedido

`GestorAutenticado` é uma fotografia tirada no login e vive na sessão. O `ContaAtivaFilter` relê a linha do gestor em cada pedido e **reconstrói o principal**, senão retirar uma permissão não tinha efeito enquanto a pessoa não saísse e voltasse a entrar.

O filtro isenta apenas os caminhos que não pressupõem sessão válida (login, registo, logout). Não voltes a isentar o prefixo `/api/auth/` inteiro: `/estado`, `/password` e `/treinador/ligar` usam uma sessão aberta como qualquer rota protegida.

4.5 No frontend, tudo o que muda dados chama `recarregar()`

Não há estado reactivo. Enquanto cada handler decidia o que refrescar, faltava sempre um pedaço: houve um painel a ficar com dados velhos e um mostrador a não mexer depois de um pagamento.

Existe um `recarregar()` (ligas, detalhe, e as dívidas só se a tab estiver aberta) e todas as acções que mudam alguma coisa chamam-no. Não acrescentes refrescamentos avulso.

4.6 Nunca ponhas `-Djava.net.preferIPv6Addresses=true`

Já estive no `Dockerfile`, posto para a rede privada do Railway, que só existe em IPv6. Estava a mais e fazia mal.

A opção só decide a **ordem** quando um nome tem endereços das duas famílias. Um nome que só tem IPv6, como o `postgres.railway.internal`, é usado pela JVM sem opção nenhuma. Medido, com e sem:

<code>PREFERIPV6ADDRESSES</code>	<code>API.RESEND.COM</code> (AS DUAS)	NOME SÓ COM IPV6
<code>true</code>	IPv6	IPv6
<code>system</code>	IPv4	IPv6
ausente	IPv4	IPv6

O que ela fazia era mandar para IPv6 tudo o que tem as duas, incluindo a API do Resend. O Railway não tem saída IPv6 para a internet pública, por isso os emails de recuperação morriam em `Network is unreachable`.

O sintoma apareceu longe da causa: parecia problema do Resend, ou do DNS do domínio, e era uma opção da JVM posta meses antes por outra razão. Se um dia alguma chamada a um serviço externo falhar com `Network is unreachable` em produção, começa por aqui.

5. SEGURANÇA

- **Autenticação** por sessão, com cookie `HttpOnly` e `SameSite=Lax`. Passwords em BCrypt.
- **Autorização por dono, não por perfil.** Quase tudo é isolado pelas consultas: `buscarPorIdEGestor` faz a verificação acontecer dentro do SQL. As únicas excepções por papel são `/api/admin/**` (`ROLE_ADMIN`) e `criar liga` (`PODE_CRIAR_LIGAS`).
- **CSRF** com cookie `XSRF-TOKEN` legível por JavaScript, reenviado no cabeçalho `X-XSRF-TOKEN`. Obrigatório em tudo o que não seja `GET`.
- **404 em vez de 403** quando o recurso é de outra pessoa, para não confirmar que existe.
- **Contas desactivadas** são barradas no pedido seguinte, não no próximo login.
- **Sessões cortadas por mudança de password.** `Gestor.sesoesValidasDesde` é uma data de corte: o `ContaAtivaFilter` recusa qualquer sessão criada antes dela. Sem isto, mudar a password não expulsava ninguém, porque a sessão vive do lado do servidor e não sabe nada da password. Quem recupera a password costuma fazê-lo por desconfiar que alguém entrou; deixar a sessão dessa pessoa viva tornava a recuperação inútil. Também é accionada quando um administrador corrige o email de uma conta.
- **Recuperação de password** (`RecuperacaoService`): pedir responde sempre 204, exista a conta ou não, para o endereço não servir de lista de quem está registado. O código tem 24 bytes aleatórios, vale uma hora, serve uma vez, e é guardado em SHA-256 e não em claro. Máximo de três pedidos por conta por hora.
- **Convite de treinador acessível sem sessão** (`ConviteTreinadorPublicoController` e `convite.html`). Tem de ser: quem chega pelo link ainda não tem conta para entrar. O que o protege é o código — 24 bytes aleatórios, que não se adivinham — e o que devolve é o mínimo para a pessoa se reconhecer no convite: quem convidou, que equipa, que liga. Sem email nenhum, sem o código de volta, e com a mesma resposta para um código inventado, gasto, revogado ou expirado. O código viaja no endereço, como no link de recuperação: fica no histórico do browser, e é um risco aceite — o link está na conversa onde o gestor o mandou, que dura muito mais do que o histórico, e tirá-lo dali estragava o caso de quem tem de ir entrar primeiro e voltar a abrir o mesmo link.
- **Logos** são validados pelos primeiros bytes (`ImagemSuportada`), não pelo `Content-Type` que o cliente declara. SVG fica de fora, para não servirmos scripts do nosso próprio domínio.

6. BASE DE DADOS

Flyway é o dono do esquema e o Hibernate está em `ddl-auto=validate`: se as entidades não corresponderem às tabelas, a aplicação não arranca.

MIGRAÇÃO	O QUE FAZ
V1	esquema inicial: gestor, treinador, liga, equipa, jornada, resultado
V2	administração e convites
V3	logo por liga
V4	contas de treinador
V5	dívidas, blocos e regra
V6	permissão de criar ligas
V7	marca de jornada já incluída num bloco
V8	recuperação de password por email
V9	convite ao lugar: <code>equipa.treinador_id</code> único e <code>convite_treinador.equipa_id</code>
V10	email do treinador e rasto do envio do convite
V11	um convite vivo por lugar (índice único) e prazo nos que não tinham
V12	escala por tabela, treino cobrado ou não, pontos de treino na classificação
V13	cobranças de época (Inverno/Verão) e o nome no bloco da dívida
V14	versão na jornada, para dois fechos simultâneos não cobrarem a dobrar

A V9 verifica os dados antes de apertar o esquema e **falha com mensagem** se encontrar um treinador partilhado por duas equipas ou um convite sem equipa — casos que a aplicação não produz, mas que só quem conhece os dados pode resolver. Como o PostgreSQL faz DDL transaccional e o Flyway corre cada migração numa transação, uma falha destas não deixa nada a meio.

Nunca edites uma migração já aplicada em produção. O Flyway guarda o checksum e recusa arrancar se ele mudar. Acrescenta uma migração nova.

Antes de aplicar uma migração em produção, faz cópia e confirma que ela restaura:

```
export DATABASE_URL='postgresql://...'          # o público, do painel
./scripts/backup-railway.sh
./scripts/backup-railway.sh --verificar backups/ligadospobres-*.sql.gz
```

A verificação restaura para uma base descartável e conta o que lá ficou. Ler o índice do ficheiro (`pg_restore -list`) diz que ele não está truncado; só o restauro diz que ele presta.

Nada apaga ligas nem equipas: quem sai fica marcado como desistente e o histórico mantém-se.

7. FRONTEND

Sem passo de build, sem framework, sem dependências externas. A Content-Security-Policy é `default-src 'self'` e até os tipos de letra são servidos pela aplicação, para não haver exceções.

Cada página duplica os seus próprios utilitários (`api()`, `mostrarAlerta()`, `tokenCsrf()`, `texto()`). É uma decisão, não um esquecimento: sem bundler, partilhar significaria mais um pedido e mais uma ordem de carregamento para acertar.

Escapa sempre o que vem do servidor com `texto()` antes de o meter em HTML.

Páginas: `index.html` (gestor), `minhas-ligas.html` e `minhas-dividas.html` (treinador), `admin.html`, `login.html`, `registo.html`, `registo-treinador.html`, `convite.html`, `conta.html`.

A `convite.html` é a única que decide o que mostrar depois de falar com o servidor: lê o convite pelo código do endereço e, conforme haja ou não sessão aberta, oferece criar conta ou juntar a equipa à conta que já existe.

Folha de estilos

Tema escuro, com um sistema de tokens no topo do `styles.css` (cores, escala tipográfica, sete valores de espaçamento). Um só tipo de letra, a Outfit, alojada na própria aplicação (`static/fontes/`) para a CSP poder continuar a ser `default-src 'self'`.

Aqui estive o par Archivo + Archivo Narrow e a hierarquia fazia-se com duas larguras. A Outfit tem uma largura só, por isso o que separa um rótulo de uma frase passou a ser o tamanho, as versaletes e o espaçamento entre letras. Como é geométrica e mais larga em maiúsculas, todo o `letter-spacing` dos rótulos foi encolhido cerca de um terço: manter o que a Narrow levava punha as etiquetas a rebentar as colunas.

Há **um só acento**, o laranja `--sinal`, e usa-se apenas onde carrega informação: o líder da classificação, o separador activo e o foco do teclado. Não o uses para mais nada, ou deixa de querer dizer alguma coisa. Para estados há cores próprias: `--ok` verde, `--espera` âmbar, `--fim` vermelho, `--frio` azul.

8. TESTES

```
mvn test
```

224 testes, todos ao nível do serviço ou do domínio, com os repositórios em memória. **Não há testes de controller**, por convenção: a lógica está nos serviços e é lá que é testada.

`ArranqueTest` levanta o contexto Spring com Testcontainers, por isso precisa de Docker. Para correr sem ele:

```
mvn test -Dtest='!ArranqueTest'
```

Testes de fluxo, ponta a ponta

`testes-e2e/` são guiões em Python que falam com a aplicação a correr, por HTTP. O `mvn test` cobre os serviços; estes cobrem o que só aparece com a aplicação de pé — sessões, autorização entre contas, concorrência e o que sobrevive a um reinício. Correm contra uma base descartável e repetem-se de propósito: cada corrida usa um sufixo aleatório, e é a repetição contra a mesma base que apanha os erros de "segunda vez". As instruções estão no `testes-e2e/LEIA-ME.md`.

Foi assim que apareceram três problemas que os testes de serviço não viam: o fecho simultâneo da mesma jornada a cobrar a dobrar (ver `Jornada.versao`), a grelha de jornadas e dívidas a pôr a página a rolar de lado no telemóvel, e um texto comprido de mais a sair como 409 "isto foi alterado por outro pedido ao mesmo tempo" — uma mensagem sobre uma corrida, para quem só escreveu um nome grande. Os limites das colunas passam a ser verificados do lado de cá (`RegrasDeConta.textoValidado`), com a mensagem a dizer qual é o campo e qual é o limite.

Como testar em condições

A base de dados de desenvolvimento pode ter credenciais desalinhadas com o `.env`, porque a password só é aplicada quando o volume é criado. Para provar comportamento ponta a ponta, o mais rápido é levantar um ambiente descartável:

```
docker run -d --name qa-db -e POSTGRES_DB=qa -e POSTGRES_USER=qa \
-e POSTGRES_PASSWORD=qa123 -p 55432:5432 postgres:18

DB_URL=jdbc:postgresql://localhost:55432/qa DB_USER=qa DB_PASSWORD=qa123 \
ADMIN_EMAIL=qa@teste.local ADMIN_PASSWORD=qalocal12345 mvn spring-boot:run
```

Vale a pena: bugs de refrescamento da interface e de estado só aparecem assim.

9. DEPLOY

`master` faz deploy automático no Railway. O jar é único e serve também o frontend.

```
railway status          # estado do serviço
railway logs            # ver o arranque
```

Confirma sempre nos logs que as migrações validaram e que o arranque foi limpo. Confirma também que não aparece o aviso `Sem EMAIL_CHAVE` ou `EMAIL_REMETENTE`: se aparecer, as mensagens vão para o log em vez de serem enviadas — ninguém consegue recuperar a password, e os convites de treinador têm de ser entregues à mão pelo link.

Email

Três variáveis de ambiente:

VARIÁVEL	PARA QUE SERVE
<code>EMAIL_CHAVE</code>	Chave de API do Resend
<code>EMAIL_REMETENTE</code>	Remetente, num domínio verificado no Resend
<code>APP_URL</code>	Endereço público, usado nos links dos emails

Duas mensagens: a recuperação de password e o convite de treinador.

O `EMAIL_REMETENTE` tem mesmo de ser de um domínio verificado no Resend, com os registos de DNS configurados. Com o remetente de teste deles, só a caixa de correio do dono da conta recebe as mensagens, o que na prática deixa a recuperação sem funcionar para toda a gente menos uma pessoa.

Sem chave nenhuma a aplicação arranca à mesma e o `EnviadorParaLog` escreve as mensagens na consola, com o link e tudo. É assim que o fluxo se experimenta em desenvolvimento.

`EnviadorDeEmail.enviar` devolve se a mensagem saiu, e o `EnviadorParaLog` devolve `false` — porque é a verdade, nada saiu. A recuperação de password ignora o resultado de propósito (a resposta é a mesma exista ou não a conta, ou o endereço passava a dizer quem tem conta aqui); o convite de treinador usa-o para dizer ao gestor que o email não foi enviado e que entregue o link. Em produção isto só acontece com a chave por configurar, e é assim que se dá por isso sem ler os logs.

O HTML das mensagens

Está todo no `ModeloDeEmail`, e parece antiquado de propósito. HTML de email não é HTML de página: tabelas para a disposição, estilos em linha, sem `<style>`, sem imagens e sem os tipos de letra da aplicação, porque um `@font-face` não carrega em cliente de email nenhum. O botão é uma célula de tabela com cor de fundo e não um `<a>` com padding, senão o Outlook, que desenha com o motor do Word, mostrava um link azul sublinhado onde devia estar um botão laranja.

Cada mensagem leva sempre as duas versões, texto e HTML. Quem recebe em texto simples tem de conseguir recuperar a password na mesma, e um email só com HTML é olhado de lado pelos filtros de spam.

O nome de quem recebe é escrito pela própria pessoa no registo e vai para dentro de HTML, por isso é escapado. Há um teste para isso.

O túnel para a base de dados (`railway connect Postgres --tunnel-only`) falha com alguma frequência e não há URL pública alternativa. Quando isso acontecer, reproduzir localmente costuma responder melhor à pergunta do que espreitar os dados de produção.

10. POR IMPLEMENTAR

- Verificação de email no registo. O convite já trava o registo aberto, que é o que a confirmação costuma proteger; o que fica por resolver é o email mal escrito, e para esse o remendo é o administrador corrigi-lo à mão.

